

1 Comprendre numpy

Numpy est un package python qui permet de manipuler des tableaux multidimensionnels. Il est très utilisé dans le domaine scientifique et de l'apprentissage automatique. Il est très rapide car il est écrit en C et utilise des algorithmes optimisés pour les opérations sur les tableaux.

Dans tout ce qui suit, nous importerons numpy de la façon suivante :

```
import numpy as np
```

1.1 Création d'un tableau

Il existe plusieurs fonctions pour créer un tableau numpy : `array` (création à partir d'une liste), `zeros`, `ones`, `arange` (même comportement que `range`), `linspace`, `logspace`. On peut aussi créer des tableaux aléatoires : `np.random.rand`, `np.random.normal`.

Donner un exemple de chacune de ces fonctions.

1.2 Exemples de manipulation de tableaux

- Avec numpy, calculer $\sum_k \frac{1}{k^2}$ de 1 à 100 000 et comparer à la valeur théorique de la limite qui est $\pi^2/6$. On utilisera la fonction `np.arange` (en commençant le tableau à 1) et `np.sum`. Ce calcul doit se faire sans boucle `for` en une seule ligne.
- Avec numpy et matplotlib, tracer la courbe $y = 3x - 2 \sin(5x)$ pour x entre -3 et 3.

```
import matplotlib.pyplot as plt
# plt.plot(x, y)
```

1.3 Indexer un tableau

On peut indexer un tableau numpy de plusieurs façons : avec un entier, une tranche (slice que l'on fait avec les `:`), un tableau d'entiers ou un tableau de booléens

Sur le tableau ci-dessous :

- Extraire avec un indice simple le nombre `0.14665455`
- Extraire `0.90215908`, l'avant-dernier élément.
- Extraire avec une tranche `0.31128493`, `0.14665455`, `0.09377272` et `0.14665455`, `0.94734078`, `0.45876301` (un sur deux).
- Extraire les éléments a_i tels que b_i soit vrai ? tels que b_i soit faux ? tels que $a_i > .6$.
- Créer le tableau $d_i = a_{c_i}$.
- Créer le tableau $e_i = a_{i+1} - a_i$.

```
a = np.array([0.18671948, 0.31128493, 0.14665455, 0.09377272, 0.94734078,
              0.40556268, 0.45876301, 0.26474838, 0.90215908, 0.11927916])

b = np.array([ True,  True,  True, False, False,  True,  True,  True,  True,
              True])

c = np.array([1, 3, 5])
```

1.4 Opérations sur les tableaux

- On considère $N = 1\,000\,000$ points tirés aléatoirement sur une distribution uniforme entre 0 et 1. Calculer la moyenne, l'écart type, le maximum et le minimum de cette distribution.
- On considère $N = 1\,000$ points tirés aléatoirement sur une distribution uniforme entre 0 et 1. La moyenne, l'écart type, le maximum et le minimum de cette distribution sont des variables aléatoires. Calculer l'écart type de ces variables aléatoires en effectuant l'opération $M = 100\,000$ fois. On commencera par créer un tableau de taille $N \times M$ contenant tous les tirages. Il faudra utiliser l'option `axis` des fonctions/méthodes numpy.

```
a = np.random.rand(1000, 1000000)
```

1.5 Types de données

- On considère le code suivant. Que se passe-t-il ? Pourquoi ?

```
a = np.array([1, 2, 4])
a[1] = 3.14
print(a)
```

- Calculer 3^{523} en utilisant python natif (`a = 3`) et un tableau numpy `a = np.array([3])`. Que se passe-t-il ? Pourquoi ? Quel est le type de données dans le tableau ? Comment calculer approximativement 3^{523} avec numpy ?

1.6 Conditions sur un tableau

On souhaite vérifier que $\cos^2 + \sin^2 = 1$ en prenant avec 10000 points aléatoirement entre 0 et 2π . Pourquoi le code suivant ne fonctionne pas ? Comment le faire fonctionner !

```
import numpy as np
from math import *

x = np.random.rand(0, 1)*2*np.pi

if cos(x)**2 + sin(x**2)==1:
    print('Le test fonctionne')
else:
    print('Ça ne fonctionne pas')
```

1.7 Vitesse d'exécution de numpy

On considère le code suivant qui crée un tableau et une liste d'un millions de nombre aléatoire entre 0 et 1. En utilisant la fonction `%timeit`, combien de temps faut-il pour calculer `a**2` le tableau contenant les carrés des éléments de `a` ? Justifier l'ordre de grandeur de ce nombre. Écrire une fonction python qui à partir d'une liste renvoie une nouvelle liste contenant les carrés des nombres. Calculer les temps d'exécution de cette fonction ? Commenter.

```
a = np.random.rand(1_000_000)
b = list(a)
```

```
%timeit a**2
```

1.8 Polymorphisme et fonction compatible avec les tableaux numpy

Le polymorphisme est la possibilité pour une fonction d'utiliser des arguments de types différents.

- On considère la fonction suivante. Donner au moins 5 types de données pour lesquels cette fonction ne donne pas d'erreur ?

```
def f(x, y):
    return x + y
```

- On considère les fonctions suivantes. Pourquoi la première fonctionne sur des tableaux et pas la seconde ? Que fait la fonction `np.vectorize` ? Comment l'utiliser pour résoudre le problème ? Est-ce rapide ?

```
def gaussienne(x):
    return np.exp(-x**2/2)

def triangle(x):
    if x < -1:
        return 0.
    elif x < 0:
        return 1+x
    elif x < 1:
        return 1-x
    else:
        return 0.

x = np.linspace(-2, 2)
print(gaussienne(x))
print(triangle(x))
```

1.9 Tri d'un tableau

- Comment trier un tableau ? Est-ce que le tri se fait en place ?
- Que fait la méthode `argsort` ?
- Voici une liste d'élève et de notes. Créer un tableau avec les élèves par ordre décroissant des notes

```
liste_eleves = [
    "Maël", "Rose", "Louis", "Alba", "Alice",
```

(continues on next page)

```

    "Raphaël", "Jules", "Adam", "Emma", "Léo",
    "Léon", "Gabriel", "Jade", "Noah", "Ambre",
    "Arthur", "Romy", "Adèle", "Louise", "Alma"
]

listes_notes = np.random.rand(len(liste_eleves))*20

```

1.10 Tableau numpy et espace mémoire

On considère le code suivant. Justifier la sortie !

```

a = np.array([0.18671948, 0.31128493, 0.14665455, 0.09377272, 0.94734078,
              0.40556268, 0.45876301, 0.26474838, 0.90215908, 0.11927916])
b = a[1:3]
c = 3*a
d = a[a<.8]
e = a[[1, 4, 6, 3, 4, 6]]
def f(a):
    a[6] = 9.3
    return a

a[0] = 1.2
b[1] = 4.5
c[2] = 5.6
d[3] = 3.5
e[5] = 7.4
f(a)
print(a)

```

2 Formule de Simpson

On rappelle la formule de Simpson pour le calcul approché d'une intégrale :

$$\int_a^b f(x)dx \approx \Delta_x \sum_{i=0}^{N-1} \frac{f(x_i) + 4f(x_i + \frac{\Delta_x}{2}) + f(x_i + \Delta_x)}{6} \equiv I(f; a, b, N)$$

où $\Delta_x = \frac{b-a}{N}$ et $x_i = a + i\Delta_x$.

1. Ecrivez une fonction `simpson_slow` qui calcule l'intégrale d'une fonction f entre a et b avec N pas avec la méthode de Simpson en utilisant une boucle (for loop).
2. Ecrivez une autre fonction `simpson_fast` qui fait la même chose sans utiliser de boucle (on suposera que la fonction f est vectorisée).
3. Calculez l'intégrale de $f(x) = \frac{1}{1+x^2}$ entre 0 et 1 pour $N = 1000$ et comparez le temps entre les deux fonctions en écrivant `%timeit` avant la commande.
4. Calculez la valeur théorique I^* de l'intégrale et tracez en échelle logarithmique le residu $|I^* - I(f, 0, 1, N)|$ par rapport à N . Quelle est la vitesse de convergence de cette intégrale ?

La solution analytique de l'intégrale est : $\int_0^1 \frac{dx}{1+x^2} = \arctan(x)|_0^1 = \pi/4$

3 Volume d'une sphère

On considère un nuage de points (x, y) dans un plan 2D. Les variables x et y sont indépendantes et uniformément réparties entre -1 et 1.

1. En utilisant la fonction `np.random.rand`, créer un nuage de M points et tracer ce nuage. On pourra prendre $M = 1000$
2. Tracer dans une autre couleur les points dans un cercle de rayon 1.
3. En prenant M assez grand (par exemple 10^8), calculer la probabilité d'être dans le cercle. En déduire une estimation de la surface d'un disque de rayon 1.
4. Même question que la question 3, mais dans un espace de dimension N . Par exemple $N = 5$. On écrira une fonction.

4 Analyse de données de gravimètres.

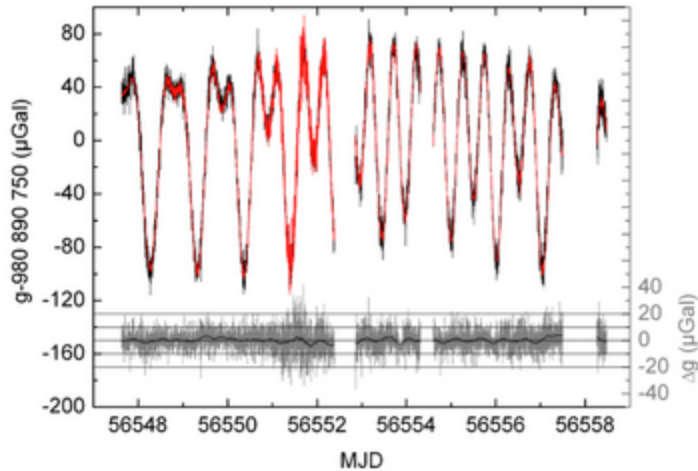
Extrait du sujet d'examen posé en 2021/2022

Un gravimètre est un appareil qui mesure l'accélération de la pesanteur. A Paris, elle vaut $g \sim 9.81 \text{ m/s}^2$. Dans cet exercice nous allons analyser des données provenant du gravimètre à atomes froids du laboratoire LNE-SYRTE (à Paris) et celles provenant d'un gravimètre à supraconducteur qui ont été enregistrées en parallèle, pendant 27 jours, entre le 7 avril et le 4 mai 2015. Le gravimètre à atomes froids (CAG, cold atoms gravimeter) est un gravimètre dit absolu dans le sens où il mesure g directement en m/s^2 . Le gravimètre à supraconducteur ('iGRAV') est un gravimètre dit relatif dans le sens où il mesure une tension en V qui peut être reliée à g . Ce dernier doit donc être calibré. Ceci peut justement se faire à l'aide du CAG. C'est ce que nous allons faire dans un premier temps.

Vous trouverez un fichier texte `data/CAGiGrav.txt` dans le dossier `data` qui contient : la date (en jour Julien modifié 'MJD'), la mesure du gravimètre (en nm/s^2 , par rapport à la valeur $g = 9.808\,907\,500 \text{ m/s}^2$), la mesure de l'iGRAV (en V) et le résidu entre les deux mesures après étalonnage de l'iGRAV (en nm/s^2). Ce contenu est séparé par des 'tabulations'. La première ligne correspond à l'en-tête.

1. Importer le fichier '`CAGiGRAV.txt`' et créer quatre tableaux de flottants (`float`) correspondant à la date, la mesure du CAG, celle de l'iGRAV et le résidu après calibration. On pourra utiliser la fonction `np.loadtxt()` dont les arguments peuvent être retrouvés à partir de sa documentation : il faudra savoir 1/ comment utiliser le `\t` comme séparateur et 2/ comment éliminer la première ligne (en-tête) du fichier.
2. Tracer l'évolution de l'accélération de la pesanteur en fonction de la date (en MJD) mesurée à l'aide du CAG et à l'aide du iGRAV. On pourra utiliser la fonction `plt.subplots` dont les arguments peuvent être retrouvés à partir de sa documentation : notamment un des arguments permettra de partager l'axe des abscisses.

Le résultat typique que l'on observe est le suivant:



Les variations de grandes amplitudes et de longues périodes sont liées à l'attraction de la lune et du soleil (phénomène des marées). Dans ce graphique, les unités sont en Gal. $1 \mu\text{Gal} = 1 \text{ nm/s}^2$.

Comme pour beaucoup de jeux de données expérimentales, il manque des données (ici soit parce que les mesures ont été perturbées par des tremblements de terre soit parce que l'asservissement des lasers (servant au refroidissement laser des atomes) du gravimètre atomique a sauté). Ces données manquantes sont représentées par des 'nan' (not a number).

La fonction `np.isnan` permet de créer un tableau de booléen contenant les nombres qui sont des 'nan'. Le code suivant permet de créer un masque (tableau de booléen) contenant uniquement les données valides en même temps pour les deux gravimètres:

```
mask = ~(np.isnan(CAG) | np.isnan(iGRAV))
```

3. Pourquoi avoir utilisé `~` et `|` au lieu de `not` et `or` ?
4. Quelle était la valeur moyenne et l'écart type de g entre le 7 avril et le 4 mai 2015 (c'est à dire sur l'ensemble des données) ?

Les données de l'iGRAV sont données en volt. On note K le coefficient de calibration de l'instrument (en $\text{nm.s}^{-2}/\text{V}$). On prendra $K = -898.25 \text{ nm.s}^{-2}/\text{V}$.

5. Calculer et tracer la différence entre les mesures enregistrées par le CAG et celles enregistrées par l'iGRAV après calibration: $\text{CAG} - K * \text{iGRAV}$. Vérifier graphiquement que votre résultat correspond bien à la quatrième colonne du fichier '`CAGiGRAV.txt`'.
6. Quel est l'écart type de ces résidus sur la période des 27 jours de mesures ? Comparer celui-ci avec l'écart type des mesures du gravimètre atomique (CAG) seules, calculées à la question 4. Commenter.

Grâce au gravimètre à supraconducteur, il a été possible d'éliminer les variations dues aux effets des marées et d'en déduire le bruit du CAG (le bruit de l'iGRAV est négligeable en comparaison). Lorsque l'on n'a pas cet appareil (l'iGRAV), il est possible de caractériser le bruit de l'instrument (le CAG) grâce à un outils mathématique appelé **la variance d'Allan**. L'idée est que entre deux mesures consécutives, le signal varie peu et que seul le bruit de l'instrument va changer le résultat.

La variance d'Allan est définie par :

$$\sigma_{\text{Allan}}^2 = \frac{1}{2} \left\langle (x_{l+1} - x_l)^2 \right\rangle_l$$

7. (facultative) Question de statistique: vérifier que si les x_i sont indépendants, σ_{Allan} correspond à l'écart type des x .
8. Écrire une fonction `allan_variance(x)` qui calcule la variance d'Allan d'un jeu de données x . Bien que ce ne soit pas exactement le cas, nous allons ici supposer que chaque mesure du CAG est réalisée pendant un temps

$\tau_0 = 177$ s. Calculer l'écart type d'Allan $\sigma_{Allan}(\tau_0)$ du jeu de données du CAG et vérifier que l'on a le même ordre de grandeur qu'à la question 6.

8. (bis) Si ce n'est pas déjà fait, écrire la fonction `allan_variance(x)` sans boucle `for`.

5 Loi de Poisson et générateur de nombre aléatoire

Cet exercice s'appuie sur un sujet de l'IPT, avec une méthode légèrement différente. Regarder : Emergent Scientist 1, 7 (2017)

Une source lumineuse illumine un photomultiplicateur. Ce dispositif envoie un pulse digital d'environ 20 ns à chaque photon qu'il détecte. La sortie du photomultiplicateur est connectée à un dispositif informatique qui permet de compter le nombre de pulses reçu pendant une durée déterminée.

Le nombre de photons qui arrive pendant une durée donnée suit une loi de Poisson, c'est à dire que la probabilité de détecter k photons est donnée par :

$$p(k) = \frac{\lambda^k}{k!} e^{-\lambda}$$

où λ est le nombre moyen de photons. Le paramètre λ sera proportionnel à la durée t_0 pendant laquelle on mesure le nombre de photons : $\lambda = \Gamma t_0$.

On rappelle que l'écart type de la loi de Poisson vaut $\sqrt{\lambda}$.

Les données sont disponibles ici : `data/100secondes_200us_count.txt`. On notera x_j cette suite.

1. Le fichier de données est enregistré sous forme d'un fichier texte. Chaque point correspond à une mesure de durée $t_0 = 200\mu\text{s}$. Lire le fichier sous forme d'un tableau d'entier:

Quel est le nombre moyen de photons reçu par seconde ?

2. Calculer l'écart type de chaque mesure et vérifiez qu'il vaut $\sqrt{\lambda}$.
3. En utilisant la fonction `numpy.unique`, avec l'option `return_counts=True`, tracez la distribution de probabilité (créer un histogramme).
4. Tracez $p(k)/p(k+1)$ en fonction de k .
5. On peut créer un générateur de bits aléatoires à partir de cette séquence : si $x_{2j} > x_{2j+1}$ alors on prend 1, si $x_{2j} < x_{2j+1}$ on prend 0, sinon on élimine le point j .

Créer une fonction `bits_aleatoires(data)` qui engendre cette suite de bits aléatoire que l'on appellera a_j - une fois avec une boucle `for` et une autre fonction sans boucle `for`. Comparez les temps.

6. On peut ensuite créer une suite de nombre aléatoire $\{b_j\}$ entre 0 et 1 en regroupant les bits $\{a_i\}$ dans une manière que b_j soit écrit en binaire comme $(a_{Nj}, a_{Nj+1}, \dots, a_{N(j+1)-1})$. On prendra par exemple $N=11$.

$$b_j = \sum_{i=0}^{N-1} \frac{a_{Nj+i}}{2^{i+1}} \in [0, 1]$$

Si $n = |\{a_j\}|$, le nombre d'éléments dans la suite $\{a_j\}$, n'est pas divisible par N , supprimez les premiers $n \% N$ éléments.

7. Si X et Y sont deux variables aléatoires ayant une distribution uniforme entre 0 et 1, alors on a

$$P(X^2 + Y^2 < 1) = \frac{\pi}{4}$$

Déterminez π en utilisant notre générateur de nombre aléatoire.